MUSIC-STAR: A STYLE TRANSLATION SYSTEM FOR AUDIO-BASED RE-INSTRUMENTATION

Mahshid Alinoori **Vassilios Tzerpos**
Department of Electrical Engineering and Computer Science, York University, Canada
{mahshida, bil}@yorku.ca

ABSTRACT

Music style translation aims to generate variations of existing pieces of music by altering the style-related characteristics of the original piece while content, such as the melody, remains unchanged. These alterations could involve timbre translation, re-harmonization, or music rearrangement. Previous studies have achieved promising results utilizing time-frequency and symbolic music representations. Music style translation on raw audio has also been investigated and applied to single-instrument pieces. Although processing raw audio is more challenging, it provides richer information about timbres, dynamics, and articulations.

In this paper, we introduce Music-STAR, the first audio-based translation system that translates the existing instruments in a piece into a set of target instruments without using source separation. To conduct our experiments, we also present an audio dataset that contains two-track pieces performed by two instrument sets alongside their stems. We carry out subjective and objective evaluations to compare Music-STAR with a variety of baseline methods and demonstrate its superiority.

1. INTRODUCTION

Music style translation is defined as transforming the style-variant components of a music piece to create variations that preserve the content. This can take several forms depending on how "music style" is characterized. Dai et al. [1] classify style into three categories: composition, performance, and timbre. Recent works have mainly focused on timbre translation [2–7] and composition style translation [8–12].

Timbre translation aims to alter the timbre information, which typically results in a change in instrumentation. Timbre translation models mostly use time-frequency representations [2–6]. Some of these [4–6] treat the spectrograms as images and apply GAN-based models [13,14] designed for image-to-image translation [15]. Timbre translation has also been explored by integrating classic signal processing and deep learning methods [16, 17].

© M. Alinoori and V. Tzerpos. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** M. Alinoori and V. Tzerpos, "Music-STAR: a Style Translation system for Audio-based Re-instrumentation", in *Proc. of the 23rd Int. Society for Music Information Retrieval Conf.,* Bengaluru, India, 2022.

The universal translation network [7] is the only model that works with audio waveforms directly. Inspired by the WaveNet autoencoder [18], it translates an arbitrary source piece to several specific timbre domains. Although one universal encoder is used to encode the domain-independent features, every domain needs to have a specific conditional WaveNet [19] decoder.

On the other hand, composition style transfer attends to tasks such as re-harmonization and music rearrangement. [1] Almost all works on composition style translation exploit symbolic representations, i.e., MIDI and piano rolls. Some of these models focus on music rearrangement by altering the accompaniments [8–10] where the style is associated with the genre. MIDI-VAE [9] is among the few cases that do not overlook the dynamics and account for note velocities. Wang et al. [11] has introduced the only GAN-based model that operates on symbolic representation to perform genre transformation. Hung et al. [12] approach music rearrangement by modifying the instrumentation, where they establish a correlation between rearrangement and transforming multiple timbres in a musical piece.

In this paper, we explore music re-instrumentation of audio waveforms that contain more than one instrument. We present several baseline solutions and then propose Music-STAR, a system designed explicitly for audio-based translation, which is built upon the WaveNet autoencoder [18]. To conduct our experiments, we also present a dataset called StarNet, in which every piece of music is performed by two different sets of instruments. The source code, audio samples, and supplementary materials are available at https://mahshidaln.github.io/Music-STAR.

2. DATASET

In order to train and evaluate Music-STAR and the baseline models, we need an audio dataset that contains multi-track pieces played with different sets of instruments alongside their stems. For this purpose, we have created the StarNet dataset, in which every piece is composed of two instrument tracks from two domains: strings-piano and clarinet-vibraphone. In other words, for every piece, the dataset includes strings-piano and clarinet-vibraphone mixtures as well as their corresponding isolated tracks.

We have chosen piano ↔ vibraphone and strings ↔ clarinet translations to preserve the type of excitation in

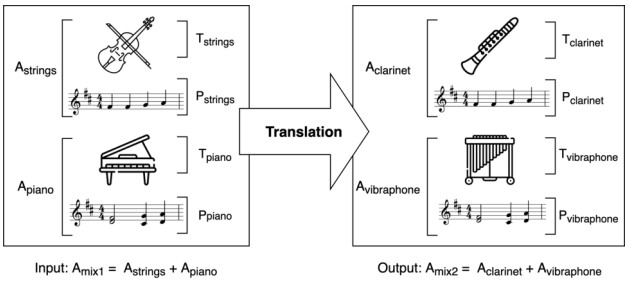


Figure 1. An example of multi-instrument translation where the timbre components of strings ($T_{strings}$) and piano (T_{piano}) are translated into clarinet ($T_{clarinet}$) and vibraphone ($T_{vibraphone}$), respectively, while retaining the corresponding pitch components ($P_{strings} = P_{clarinet}$ and $P_{piano} = P_{vibraphone}$).

the source/target instruments (discrete excitations in piano and vibraphone versus continuous excitations in clarinet and strings).

Accessing recorded music stems is often hard due to copyright limitations. Therefore, we select a variety of music pieces from MusicNet [20] and other freely available classical music MIDI collections. Instead of using their original instruments, we apply virtual instruments for the aforementioned combinations. The resulting dataset contains two domains, one for each instrument combination, adding up to roughly 11 hours of audio for each domain. The StarNet dataset is available at `https://zenodo.org/record/6917099`.

To conduct our experiments, we use two versions of StarNet:

1. Preprocessed StarNet: The preprocessed version of StarNet includes uncompressed stereo WAV files with a sample rate of 44.1 kHz and a bit depth of 16 bits. The preprocessing step includes detecting and removing the intervals where one or both instruments are silent to ensure their simultaneous presence for a considerable amount of time. After the silence removal, the dataset size shrinks to roughly 9 hours. Silence detection is done by indicating the minimum loudness and minimum silence duration.

2. Reduced StarNet: The reduced version of StarNet is obtained after resampling the preprocessed version at 16 kHz, merging the two audio channels and outputting mono audio, and finally quantizing the audio by 8-bit mu-law encoding.

3. METHODOLOGY

Pitch and timbre are among the fundamental acoustic properties of every audio track in a recorded mixture. As we address instrumentation changes in our work, timbre is regarded as the style component, while pitch constitutes the content we intend to preserve. Based on this definition, we introduce the following notation of a piano track and its style and content components:

$$A_{piano} := T_{piano} + P_{piano}$$

where A is an audio signal, T is the timbre component, and P is the set of pitch components and their embodied durations in that signal. Separating style and content components, also known as disentanglement, has been leveraged in previous music translation studies [7,9,12] and is mostly addressed by adversarial learning in encoder-decoder architectures. Note that T and P in the notations are conceptual terms, and the corresponding features will be extracted by the autoencoders and represented by the embeddings.

An example of single-instrument translation is shown below, where a piano track is translated to vibraphone in a way that the pitch component is retained, and the timbre component alters:

1. $Input = A_{piano} := T_{piano} + P_{piano}$
2. $Output = A_{vibraphone} := T_{vibraphone} + P_{piano}$

Our task is to tackle a more complex problem, i.e., dealing with multi-instrument pieces (Fig. 1). In the case of our dataset, the input audio signal will be one of the following:

1. $A_{mix1} = A_{strings} + A_{piano} :=$
   $T_{strings} + P_{strings} + T_{piano} + P_{piano}$
2. $A_{mix2} = A_{clarinet} + A_{vibraphone} :=$
   $T_{clarinet} + P_{clarinet} + T_{vibraphone} + P_{vibraphone}$

As a result, pitch-timbre disentanglement does not suffice here as we need to isolate two sets of timbre and pitch components.

In this section, we first introduce the baseline methods for performing multi-instrument translation, i.e., single-instrument translation pipeline and separation-based translation pipeline. Then we present our proposed methods, i.e., the embedding-supervised method and Music-STAR.

3.1 Single-instrument Translation Pipeline

The most simplistic approach to multi-instrument translation is to translate each instrument track separately and obtain the final output by mixing them. However, this is only possible when the music stems are available.

In order to implement this method, we employ an existing audio-based translation model [7], which is built upon the WaveNet autoencoder [18]. This model consists of a universal encoder and six decoders corresponding to six target domains, all collected from the MusicNet [20] dataset.

In this universal network, the temporal encoder maps the pitch components into an embedding space utilizing a WaveNet-like architecture. The decoders are conditioned on the pitch embeddings and generate audio samples that entail the specific timbre they have been trained for in an autoregressive manner. A domain confusion network is employed during training to ensure that no domain-specific information is included in the embeddings by imposing domain confusion loss [21]. The authors also emphasize the role of distorting the input by modulating the pitch locally

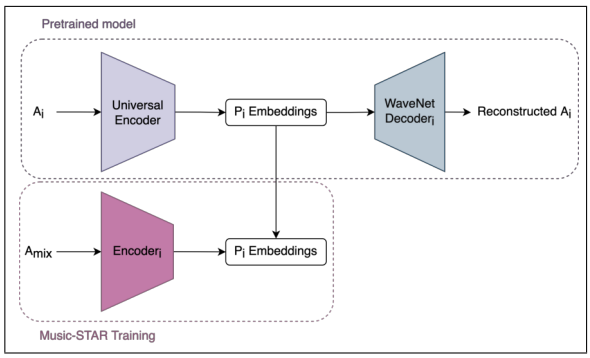


Figure 2. The embedding-supervised encoder is trained to generate the same embeddings as the universal encoder for every instrument *i* present in an audio mix (A_{mix}).

in every audio segment while training. This way, they ensure that the encoder does not memorize the content and drive it to capture meaningful features. In other words, the network is trained as a denoising autoencoder that learns to recover the original input by applying teacher forcing [22] while training the decoders. The teacher forcing technique is carried out by feeding the original input audio segment to the decoder instead of the generated samples.

3.2 Separation-based Translation Pipeline

The previous method does not apply to use cases where the instrument tracks are not available separately and where the system must take the audio mixture as input. One way to tackle this is to adopt a pipeline of audio source separation and single-instrument translation. The source separation module isolates each of the tracks so that a single-instrument translation model can transform each of them into a target instrument.

To perform source separation, we adopt Demucs [23], a state-of-the-art music source separation model that operates in the waveform domain, taking an audio mixture and outputting isolated audio tracks. These stems are then fed into the universal network described in Section 3.1 for the translation phase.

3.3 Embedding-supervised Method

Training source separation models to separate every instrument in a mixture is a demanding task, and thus we favor solutions that do not depend on it. We have investigated a potential solution that we refer to as the embedding-supervised method, which performs a semi-separation task through the encoding process. In this case, the embedding-supervised encoder learns to capture the pitch-related features of one of the two instruments in the mixture. In order to achieve this, we need a pre-trained encoder that can provide the desirable pitch embeddings for a single instrument and assist the embedding-supervised encoder in distinguishing between the instruments throughout the training step. Recall that the universal encoder in our baseline model learns to extract pitch-related information from the input. Thus the output of such an encoder can provide the embeddings we seek. Based on this, we can train the embedding-supervised encoder to mimic the output of the universal encoder when given a mixture as the input. (Fig. 2)

For instance, if we have the mixture as:

$$A_{mix} = A_{strings} + A_{piano}$$

we feed the piano track A_{piano}:

$$A_{piano} := T_{piano} + P_{piano}$$

into the universal encoder, and its output will represent piano track pitch information (P_{piano}). We then input the mixture A_{mix} to the embedding-supervised encoder and train it to output the same code, i.e., the embeddings for P_{piano}. Consequently, we have a piano-specific encoder to extract the piano pitch content from any mixture. Note that for such training, existence of the stems in the dataset is necessary. We can isolate the information corresponding to one instrument via this approach without applying actual source separation. We can train one encoder per instrument with the help of the universal encoder. The model strives to minimize the loss function below:

$$L(E^i_{es}(mix), E_u(x^i)) \tag{1}$$

where E_{es} is the embedding-supervised encoder, mix is the audio segment from the input mixture, E_u is the universal encoder, x^i is the fragment of the instrument track i that we want to extract from the mixture, and L is L1 loss.

During inference, a pre-trained WaveNet decoder can also be engaged to translate the code to an arbitrary target instrument.

3.4 Music-STAR

Our ultimate goal is to realize the idea of multi-instrument translation where we do not need to engage source separation modules or additional encoders, and in general remove the restrictions of the aforementioned techniques. To this end, we introduce Music-STAR, designed explicitly for audio-based multi-instrument translation as described below.

As mentioned in Section 3.1, the universal network is trained using a random local pitch modulation that distorts the input to ensure that the encoder does not memorize the input content. Since the encoder learns to extract pitch-related information, we expect the code to embody data from the distorted segments. However, the decoder is trained using the teacher forcing technique, where the original audio segment is fed into the decoder alongside the embeddings produced by the encoder. The benefits of this approach are two-fold. First, the decoder learns the timbre of the original segment that is not present in the latent space. Second, for the decoder to reconstruct the original audio with the right pitch, it forces the encoder to represent the original segment in the latent space by removing the distortion. We will employ a similar idea below.

To begin, we represent our input audio as:

$$A_{input} = A_{strings} + A_{piano} := \\ T_{strings} + P_{strings} + T_{piano} + P_{piano}$$

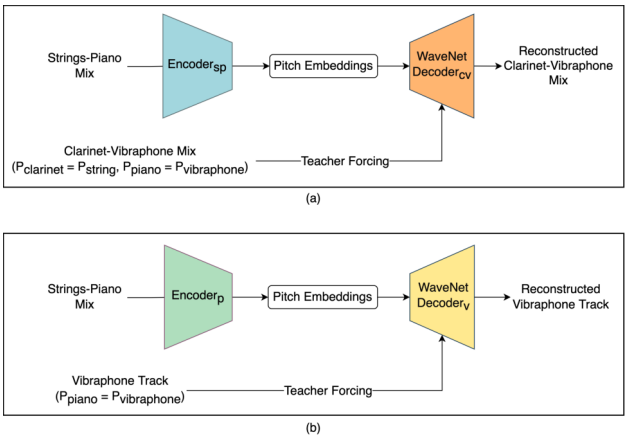


Figure 3. Training Music-STAR using (a) the mixture-supervised method to generated the target mixture, (b) the stem-supervised method to generate a single target stem. The modules subscripts *sp*, *cv*, *p*, and *v* corresponds to string-piano, clarinet-vibraphone, piano, and vibraphone, respectively.

The output we are looking for is:

$$A_{output} = A_{clarinet} + A_{vibraphone} := \\ T_{clarinet} + P_{clarinet} + T_{vibraphone} + P_{vibraphone}$$

where $P_{strings} = P_{clarinet}$ and $P_{piano} = P_{vibraphone}$.

To obtain this output, we should make sure that:

1. The encoder includes only the representations of P_{piano} and $P_{strings}$ in the embeddings, removing the information related to the other components of A_{input} (T_{piano} and $T_{strings}$), which has been proven possible when removing the distortion in [7].

2. The decoder generates the output signal by applying the target timbres that it has learned from the audio segments used for teacher forcing.

Accordingly, we conclude that the autoencoder will be able to extract the pitch-related features of the input mixture and translate it into target timbres if we train the model by:

1. Using the strings-piano mixture as the encoder's input, and

2. Using the clarinet-vibraphone counterpart of the input to apply teacher forcing to the decoder.

When the decoder is learning to translate the encoder's output to generate the clarinet-vibraphone segment, it inevitably guides the encoder to hand in helpful information for the task, which should be strings and piano pitch representations (see Fig. 3(a)). Since we train the model using the source and target audio mixtures, we name this approach the *mixture-supervised* method. The loss function for the mixture-supervised method is formulated as

$$\sum_{j} L(D(E(mix), t_j), t_j) \qquad (2)$$

where L is the cross-entropy loss, D is the decoder, E represents the encoder, mix is the input mixture segment, and t_j is the jth sample from the target mixture used for teacher forcing.

In our experiments, we also account for a variation of the mixture-supervised method where the target mixture (used in teacher forcing) is replaced by only one of its instrument tracks. We call the resulting method *stem-supervised*, in which we employ two autoencoders, each for translating one of the instruments in the mixture, and combine their outputs in the end to obtain the final mix (see Fig. 3(b)).

4. EXPERIMENTS

4.1 Single-instrument Translation Pipeline

We employ the universal network [7] to perform single-instrument translation. The network is originally trained on six classical music domains from the MusicNet dataset that includes mono audio segments with a sample rate of 16 kHz, which are later quantized by 8-bit mu-law encoding. The input files are randomly selected, and then a 0.75-second audio chunk is randomly segmented out of the file for every training input. A duration between 0.25 to 0.5 seconds of that segment is then selected for applying distortion by modulating the pitch.

In order to use this architecture as a baseline, we need to make sure that the decoders can generate the timbres included in the StarNet dataset. Therefore, we use the pre-trained universal encoder and finetune the decoders on the reduced StarNet dataset. Note that the data in reduced StarNet has the same properties as the data used for training the original model in terms of sample rate, bit depth, and the number of channels. Since the universal encoder has been trained using substantial computational resources on six domains and captures domain-agnostic features, it is powerful enough to successfully encode inputs from other domains. We finetuned the decoders with a batch size of 16, a learning rate of $1\mathrm{e}{-3}$, and a decay factor of 0.98 for 100 epochs.

4.2 Separation-based Translation Pipeline

We adopt Demucs [23] as the music source separation model, which is trained originally on MuseDB [24]. It is vital for our experiment that the network separates the two instruments present in the mixtures. Therefore, we train Demucs on preprocessed StarNet from scratch. We use the original learning rate of $3\mathrm{e}{-4}$ and Adam optimizer with a batch size of 32 for 250 epochs. The resulting model can take an arbitrary length of an input mixture and successfully separate the strings track from the piano or the clarinet track from the vibraphone.

The separated audio tracks are then post-processed to comply with the data configuration in reduced StarNet and fed into the finetuned models from Section 4.1 that correspond to their target instruments.

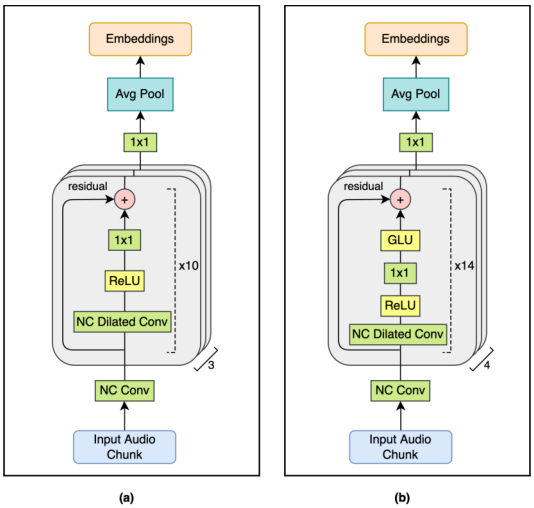


Figure 4. (a) The architecture of the universal encoder. (b) The architecture of the encoders used in the embedding-supervised and Music-STAR models.

4.3 Embedding-supervised Method

An autoencoder architecture builds the backbone of the model. This method focuses on training the embedding-supervised encoder with the help of the universal encoder. The architecture of the universal encoder is depicted in Fig. 4(a). Since the embedding-supervised encoder should learn to extract the same information as the universal encoder from a mixture rather than a stem, one would expect that a more complex architecture would be required for it. This was indeed confirmed by our pilot experiments.

The architecture we use as the embedding-supervised encoder is shown in Fig. 4(b). Similar to the universal encoder, it starts with a non-causal non-dilated convolution. The encoder consists of four blocks of 14-layer residual networks composed of non-causal dilated convolutions with a kernel size of 3, followed by ReLU nonlinearity and a 1x1 convolution. A GLU activation function follows the 1x1 convolution, and the output is summed with the input to form the residual connection. The result of this connection is then fed into the next layer of the encoder. The dilation increase factor of 2 and the 128 channels are identical to the universal encoder.

Unlike the baseline model that trains a universal encoder for all the domains in the training set, we need to train a single encoder for each instrument to extract content information from a mixture. The embedding-supervised encoders are trained on 0.75-second audio chunks randomly segmented out of the mixture files from reduced StarNet, and the same segments are extracted from the instrument tracks to be fed into the universal encoder. We adopt Adam optimization and exponential learning decay with a learning rate of $3e{-}4$, a decay factor of 0.98, and a batch size of 16 for a total of 100 epochs.

The finetuned WaveNet decoders described in Section 4.1 are then attached to the embedding-supervised encoders during inference to translate the code into the target instruments.

4.4 Music-STAR

We employ the WaveNet autoencoder architecture in the mixture-supervised method as well. The encoder we use for this method has the same architecture as the one used in the embedding-supervised method (Fig. 4(b)). The decoder's architecture is identical to the WaveNet decoders described in [7].

We train the models on the reduced version of StarNet. We pick random one-second audio segments of the input mixture and extract the same segment in the target mixture to apply teacher forcing to the decoder. Training is done using the Adam optimizer and exponential learning decay, where a learning rate of $3e{-}4$ and a decay rate of 0.99 are applied. The model is trained for a total of 100 epochs with a batch size of 16.

We use the same training configuration as above for the stem-supervised setting, except that two autoencoders are involved, each for transforming the source mixture into one of the target instruments.

5. EVALUATION

We conduct subjective and objective evaluations of the obtained audio and compare the re-instrumentation methods on three criteria:

1. Content preservation: how much of the pitch content of the input is retained in the output.
2. Style fit: How well the output presents the target timbres.
3. Audio quality: How clean and distortion-free the generated audio is.

5.1 Subjective Evaluation

The subjective evaluation provides a qualitative assessment of our models based on how users perceive the generated outputs. We carry out the subjective evaluation by distributing a survey among 30 participants, some of whom have a musical background. Each survey contains two different pieces, one from each domain (strings-piano and clarinet-vibraphone) selected out of 10 pieces in total. The target mixture is provided for each piece to demonstrate the gold standard, followed by corresponding translation outputs resulting from the five methods. The order of outputs is different for the two pieces, and the participants have no prior knowledge of the order.

Every piece in the survey is evaluated through three questions asking the participants to rank the five outputs based on:

1. How well they preserve the target musical content, which accounts for content preservation.
2. How well they present the instruments' tone colors (timbre) of the target piece, corresponding to style fit.
3. How clean and distortion-free they are, presenting the audio quality.

Method	Subjective			Objective	
	Content	Style	Quality	Content (Jaccard)	Style (Cosine)
Single-instrument	166	150	153	0.371	0.483
Separation-based	165	147	159	0.392	0.474
Embedding-supervised	161	157	149	0.350	0.472
Stem-supervised	154	190	183	0.323	**0.699**
Mixture-supervised	**254**	**256**	**256**	**0.426**	**0.698**

Table 1. Evaluation results on the three criteria of content preservation, style fit, and audio quality. The results from the subjective evaluation show the scores for each model according to the rankings provided by the surveys. Objective evaluation reports Jaccard similarity and cosine similarity for the first two criteria, respectively.

After collecting the surveys, we concluded the rankings for each question by scoring the models. For each criterion, the methods receive a score of 5 if one of their generated outputs is ranked first, a score of 4 if ranked second, a score of 3 if ranked third, a score of 2 if ranked fourth, and a score of 1 if ranked last. The aggregate scores are shown in Table 1. More detailed analysis is available online [1] .

5.2 Objective Evaluation

We aim to provide a quantitative quality assessment on content preservation and style fit using techniques employed in previous studies.

5.2.1 Content Preservation

Following the work by Cífka et al. [3], we assess the models on content preservation by calculating the Jaccard similarity between the pitch contours of the outputs and their corresponding gold standards. Since we are addressing multi-instrument music in our study, we extract the pitch contours using the multi-pitch Melodia algorithm [25] which provides the existing pitch frequencies in Hz. We round the frequency values to the nearest semitone. Then we express the similarity of the pitch sets of each time step in terms of the Jaccard index. Higher Jaccard similarity between the output and the gold standard signifies better content preservation.

5.2.2 Style Fit

We evaluate the style fit factor using the deep metric triplet network offered by Lee et al. [26]. The network consists of a backbone model, which provides embeddings for three inputs, and a triplet model that outputs a similarity score between those embeddings. The three inputs are called *the anchor*, *the positive*, and *the negative*. The network is trained to generate the embeddings in a way that the positive is closer to the anchor than the negative. During inference, a similarity score between the anchor and the other two will be reported.

Following Cífka et al. [3], we use MFCCs as the input features as they provide timbre-related information. We train the triplet network using the mixtures in the preprocessed StarNet dataset. For instance, the MFCCs of eight-second clarinet-vibraphone audio segments are used as both the anchors and the positive inputs in the training phase. At the same time, their counterparts from the strings-piano domain are regarded as negative inputs.

During inference, we presented the translation outputs from the five methods as the anchors, their corresponding gold standard as the positive, and the performance from the other domain as the negative. We report the average cosine similarity between the outputs of each translation method and their corresponding gold standards. Higher cosine similarity denotes more likeness to the target timbre and a better style fit.

5.3 Discussion

All evaluation results are presented in Table 1. The scores reported by the subjective evaluation are the total sum of the models' scores on both Clarinet-Vibraphone $\leftrightarrow$ Strings-Piano translations. Objective evaluation reports the average performance of models in translating the two domains. Both objective and subjective evaluations conclude that mixture-supervised Music-STAR is the predominant model in performing multi-instrument music translation. The mixture-supervised method outperforms the other methods in all the assessments except for objective style fit, where it reaches an almost equal cosine similarity with the stem-supervised method. An advantage that it has over the other methods is that the presence of the stems is not necessary for training the model. Mixture-supervised Music-STAR is, to the best of our knowledge, the only model capable of performing timbre translation on multiple instruments in a mixed signal.

The embedding-supervised method is, on average, the worst-performing model. The encoder used in this model is trained based on the universal encoder's outputs to extract the pitch information of one instrument out of a mixture. An imperfect ground truth places the model at the disadvantage of even more unsatisfactory performance. Also, the performance of single-instrument and separation-based translation models on different criteria are very similar.

6. CONCLUSION

This paper introduced Music-STAR, the first audio-based multi-instrument music translation system. Music-STAR tackles multi-instrument translation without applying explicit source separation to the input mixtures. We also introduce the StarNet dataset that includes two-instrument pieces performed in two domains alongside their stems. We explored a variety of possible solutions based on the WaveNet autoencoder, and finally reached a successful mixture-supervised method capable of performing simultaneous source separation and pitch-timbre disentanglement for two instruments.

Future work will target increasing the number of instrument tracks in the mixtures, and adding to the variety of instrument combinations. We also plan to develop a more ornate dataset in terms of the details on articulations and dynamics.

[1] `https://mahshidaln.github.io/Music-STAR`

7. REFERENCES

[1] S. Dai, Z. Zhang, and G. G. Xia, "Music style transfer: A position paper," *arXiv preprint arXiv:1803.06841*, 2018.

[2] A. Bitton, P. Esling, and A. Chemla-Romeu-Santos, "Modulated variational auto-encoders for many-to-many musical timbre transfer," *arXiv preprint arXiv:1810.00222*, 2018.

[3] O. Cífka, A. Ozerov, U. Şimşekli, and G. Richard, "Self-supervised VQ-VAE for one-shot music style transfer," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 96–100.

[4] S. Huang, Q. Li, C. Anil, X. Bao, S. Oore, and R. B. Grosse, "TimbreTron: A WaveNet (CycleGAN (CQT (audio))) pipeline for musical timbre transfer," *arXiv preprint arXiv:1811.09620*, 2018.

[5] D. K. Jain, A. Kumar, L. Cai, S. Singhal, and V. Kumar, "ATT: Attention-based timbre transfer," in *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2020, pp. 1–6.

[6] C.-Y. Lu, M.-X. Xue, C.-C. Chang, C.-R. Lee, and L. Su, "Play as you like: Timbre-enhanced multi-modal music style transfer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 1061–1068.

[7] N. Mor, L. Wolf, A. Polyak, and Y. Taigman, "A universal music translation network," *arXiv preprint arXiv:1805.07848*, 2018.

[8] W. T. Lu and L. Su, "Transferring the style of homophonic music using recurrent neural networks and autoregressive model," in *ISMIR*, 2018, pp. 740–746.

[9] G. Brunner, A. Konrad, Y. Wang, and R. Wattenhofer, "MIDI-VAE: Modeling dynamics and instrumentation of music with applications to style transfer," *arXiv preprint arXiv:1809.07600*, 2018.

[10] O. Cífka, U. Şimşekli, and G. Richard, "Groove2Groove: One-shot music style transfer with supervision from synthetic data," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2638–2650, 2020.

[11] J. Wang, C. Jin, W. Zhao, S. Liu, and X. Lv, "An unsupervised methodology for musical style translation," in *2019 15th International Conference on Computational Intelligence and Security (CIS)*. IEEE, 2019, pp. 216–220.

[12] Y.-N. Hung, I. Chiang, Y.-A. Chen, and Y.-H. Yang, "Musical composition style transfer via disentangled timbre representations," *arXiv preprint arXiv:1905.13567*, 2019.

[13] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2223–2232.

[14] A. Jolicoeur-Martineau, "The relativistic discriminator: a key element missing from standard GAN," *arXiv preprint arXiv:1807.00734*, 2018.

[15] Y. Pang, J. Lin, T. Qin, and Z. Chen, "Image-to-Image translation: Methods and applications," *arXiv preprint arXiv:2101.08629*, 2021.

[16] J. Engel, L. Hantrakul, C. Gu, and A. Roberts, "DDSP: Differentiable digital signal processing," *arXiv preprint arXiv:2001.04643*, 2020.

[17] F. Ganis, E. F. Knudesn, S. V. Lyster, R. Otterbein, D. Südholt, and C. Erkut, "Real-time timbre transfer and sound synthesis using DDSP," *arXiv preprint arXiv:2103.07220*, 2021.

[18] J. Engel, C. Resnick, A. Roberts, S. Dieleman, M. Norouzi, D. Eck, and K. Simonyan, "Neural audio synthesis of musical notes with WaveNet autoencoders," in *International Conference on Machine Learning*. PMLR, 2017, pp. 1068–1077.

[19] A. v. d. Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," *arXiv preprint arXiv:1609.03499*, 2016.

[20] J. Thickstun, Z. Harchaoui, and S. M. Kakade, "Learning features of music from scratch," in *International Conference on Learning Representations (ICLR)*, 2017.

[21] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," *The journal of machine learning research*, vol. 17, no. 1, pp. 2096–2030, 2016.

[22] R. J. Williams and D. Zipser, "A learning algorithm for continually running fully recurrent neural networks," *Neural Computation*, vol. 1, no. 2, pp. 270–280, 1989.

[23] A. Défossez, N. Usunier, L. Bottou, and F. Bach, "Music source separation in the waveform domain," *arXiv preprint arXiv:1911.13254*, 2019.

[24] Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, and R. Bittner, "The MUSDB18 corpus for music separation," Dec. 2017. [Online]. Available: https://doi.org/10.5281/zenodo.1117372

[25] J. Salamon and E. Gómez, "Melody extraction from polyphonic music signals using pitch contour characteristics," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 20, no. 6, pp. 1759–1770, 2012.

[26] J. Lee, N. J. Bryan, J. Salamon, Z. Jin, and J. Nam, "Disentangled multidimensional metric learning for music similarity," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 6–10.